# A*search Algorithm

1.

```
import heapq
def astar(graph, heuristic, start, goal):
    queue = [(heuristic[start], 0, start,
[start])]
    visited = {}
    while queue:
        f, cost, node, path = heapq.heappop(queue)
        if node == goal:
            print("Shortest Path:", " ->
".join(path))
            print("Total Cost:", cost)
            return
        if node in visited and visited[node] <=
cost:
            continue
        visited[node] = cost
        for neighbor, weight in graph[node]:
            new_cost = cost + weight
            if neighbor not in visited or new_cost
< visited[neighbor]:
                new_f = new_cost +
heuristic[neighbor]
                heapq.heappush(
                    queue,
                    (new_f, new_cost, neighbor,
path + [neighbor])
                )
    print("No path found!")
graph = {}
n = int(input("Enter number of vertices: "))
for i in range(n):
    node = input("Enter vertex name: ")
    graph[node] = []
e = int(input("Enter number of edges: "))
for i in range(e):
```

```
    u, v, w = input("Enter edge (node1 node2
cost): ").split()
    w = float(w)
    graph[u].append((v, w))
    graph[v].append((u, w))
heuristic = {}
print("Enter heuristic values for each vertex:")
for node in graph:
    heuristic[node] = float(input("h(" + node + ")
= "))
start = input("Enter start node: ")
goal = input("Enter goal node: ")
astar(graph, heuristic, start, goal)
```

```
Enter vertex name: G
Enter number of edges: 9
Enter edge (node1 node2 cost): S A 3
Enter edge (node1 node2 cost): S D 4
Enter edge (node1 node2 cost): A D 5
Enter edge (node1 node2 cost): A B 4
Enter edge (node1 node2 cost): D E 2
Enter edge (node1 node2 cost): B C 4
Enter edge (node1 node2 cost): B E 5
Enter edge (node1 node2 cost): E F 4
Enter edge (node1 node2 cost): F G 0
Enter heuristic values for each vertex:
h(S) = 11.5
h(A) = 10.1
h(D) = 9.2
h(B) = 5.8
h(C) = 3.4
h(E) = 7.1
h(F) = 3.5
h(G) = 0
Enter start node: S
Enter goal node: G
```

2.

```
import heapq
def astar(graph, heuristic, start, goal):
    queue = [(heuristic[start], 0, start,
[start])]
    visited = {}
    while queue:
        f, cost, node, path = heapq.heappop(queue)
        if node == goal:
            print("Shortest Path:", " ->
".join(path))
            print("Total Cost:", cost)
            return
        if node in visited and visited[node] <=
cost:
            continue
        visited[node] = cost
        for neighbor, weight in graph[node]:
            new_cost = cost + weight
            if neighbor not in visited or new_cost
< visited[neighbor]:
                new_f = new_cost +
heuristic[neighbor]
                heapq.heappush(
                    queue,
                    (new_f, new_cost, neighbor,
path + [neighbor])
                )
    print("No path found!")
graph = {}
n = int(input("Enter number of vertices: "))
for i in range(n):
    node = input("Enter vertex name: ")
    graph[node] = []
e = int(input("Enter number of edges: "))
for i in range(e):
    u, v, w = input("Enter edge (node1 node2
cost): ").split()
    w = float(w)
```

```
    graph[u].append((v, w))
    graph[v].append((u, w))
heuristic = {}
print("Enter heuristic values for each vertex:")
for node in graph:
    heuristic[node] = float(input("h(" + node + ")
= "))
start = input("Enter start node: ")
goal = input("Enter goal node: ")
astar(graph, heuristic, start, goal)
```

```
Enter vertex name: F
Enter vertex name: G
Enter number of edges: 9
Enter edge (node1 node2 cost): A B 1
Enter edge (node1 node2 cost): A C 4
Enter edge (node1 node2 cost): B C 2
Enter edge (node1 node2 cost): B D 3
Enter edge (node1 node2 cost): C E 5
Enter edge (node1 node2 cost): D G 4
Enter edge (node1 node2 cost): E G 3
Enter edge (node1 node2 cost): D F 2
Enter edge (node1 node2 cost): F G 1
Enter heuristic values for each vertex:
h(A) = 5
h(B) = 6
h(C) = 4
h(D) = 3
```

```
Enter edge (node1 node2 cost): D G 4
Enter edge (node1 node2 cost): E G 3
Enter edge (node1 node2 cost): D F 2
Enter edge (node1 node2 cost): F G 1
Enter heuristic values for each vertex:
h(A) = 5
h(B) = 6
h(C) = 4
h(D) = 3
h(E) = 3
h(F) = 1
h(G) = 0
Enter start node: A
Enter goal node: G
Shortest Path: A -> B -> D -> F -> G
Total Cost: 7.0
```